

修了試験B-2

合計点 45/52 点 ?

合格点 85%以上 (44以上/52)

0 / 0 ポイント

受験者情報の登録

登録メールアドレス *

htanaka@777.so-net.jp

御名前 *

田中 秀伯

修了試験問題

45 / 52 ポイント

✕ 単語の分散表現を獲得する代表的な方法として、カウントベース手法と *0/1
推論ベース手法がある。カウントベース手法では（ あ ）。一方、推論
ベース手法では（ い ）。

（ あ ）（ い ）に当てはまる組み合わせとして、適切なものを1つ
選べ。

- ☐ （あ）語彙数に関係なく計算量が一定である （い）シソーラス手法と同等の精度を達成できること
- ☐ （あ）単語間の共起を利用しない （い）コーパス全体の共起行列を必ず作成する
- ☐ （あ）新しい単語の追加時に再計算が必要になりやすい （い）ミニバッチによる逐次的な学習が可能である
- ☒ （あ）新しい単語を再計算なしで追加できる （い）学習済みの単語表現を更新 ✕
できない



ニューラルネットワークによる学習を通して、単語の分散表現を得るための方法としてword2vecがある。Word2vecには、単語の分散表現を学習する方法として、Continuous Bag-of-Words(CBOW)とSkip-gramの2つのニューラルネットワークが実装されている。

CBOWは（ あ ）から（ い ）を予測するのに対して、Skip-gramは（ い ）から（ あ ）を予測する。

✓ （ あ ）の選択肢について、以下のうちから適切なものを1つ選べ。 * 1/1

- ☐ コーパス全体
- ☐ 対象となる1単語
- ☒ 周辺単語
- ☐ 1つの文



✓ （ い ）の選択肢について、以下のうちから適切なものを1つ選べ。 * 1/1

- ☐ コーパス全体
- ☐ 1つの文
- ☐ 周辺単語
- ☒ 対象となる1単語



自然言語処理とは、自然言語を機械的に処理することをいうが、自然言語はそのままでは機械的に処理できないため、ベクトル表現に変える必要がある。特に「単語の意味」を的確に捉えたベクトル表現を単語の分散表現と呼ぶ。

分散表現を獲得する代表的手法にシソーラス手法、カウントベース手法、推論ベースがある。カウントベース手法の代表としては（ あ ）、推論ベース手法の代表としては（ い ）があげられる。これらは（ う ）ともよばれる「単語の意味は、周囲の単語によって形成される」という仮説に基づいた手法といえる。一般にカウントベースと比べて、推論ベースによる分散表現の獲得は、データの拡張は容易で、精度は一概には決定できないとされる。

（ い ）には、単語の取り方の違いでCBOWとSkip-gramという手法がある。（ え ）はある単語から周辺の単語を予測するモデルである。一つ前の単語、ターゲットとなる単語、一つ後ろの単語をそれぞれ $w(t-1)$, w_t , $w(t+1)$ とすると、（ え ）が最大化すべき確率は（ お ）となる。

✓ （ あ ）の選択肢について、以下のうちから適切なものを1つ選べ。 * 1/1

- ☐ WordNet
- ☒ Bag of Words
- ☐ word2vec
- ☐ Corpus



✓ （ い ）の選択肢について、以下のうちから適切なものを1つ選べ。 * 1/1

- ☐ WordNet
- ☐ Corpus
- ☒ word2vec
- ☐ Bag of Words



✓ (う) の選択肢について、以下のうちから適切なものを1つ選べ。 * 1/1

- ☐ 宝くじ仮説
- ☐ インプット仮説
- ☐ アウトプット仮説
- ☒ 分布仮説



✗ (え) (お) に当てはまる組み合わせについて、以下のうちから *0/1
適切なものを1つ選べ。

CBOW ・ $p(w_t | w_{t-1}, w_{t+1})$

☐ A

CBOW ・ $p(w_{t-1}, w_{t+1} | w_t)$

☒ B



Skip-gram ・ $p(w_t | w_{t-1}, w_{t+1})$

☐ C

Skip-gram ・ $p(w_{t-1}, w_{t+1} | w_t)$

☐ D



✕ トークンをベクトル化させる手法の1つとして、推論ベース手法の Word2Vec がある。Word2Vec は（ あ ）。

ここで、CBOW は周辺単語から中心の対象単語を予測する問題を解き、skip-gram は1つの単語からその周囲の単語を予測する問題を解く。このとき（ い ）が単語の分散表現として獲得できる。

*0/1

（ あ ）（ い ）に当てはまる組み合わせについて、以下のうちから1つ適切なものを選び。

- ☐ （あ）CBOW と Skip-gram という2つのブロックで構成された1つのNNである
（い）モデルの出力
- ☐ （あ）CBOW と Skip-gram という2つのベクトル化手法の総称である （い）モデルの出力
- ☐ （あ）CBOW と Skip-gram という2つのベクトル化手法の総称である （い）モデルの重み
- ☒ （あ）CBOW と Skip-gram という2つのブロックで構成された1つのNNである ✕
（い）モデルの重み

分散表現を獲得する手法として、Word2vecが存在する。Word2vec を学習させる際、言語という大きなデータを学習させるために、様々な高速化手法が取り入れられている。

まず入力として one-hot ベクトルを用いると語彙数の増加に連れて計算量が爆発的に増加してしまうため、単語を語彙数より低い次元の密なベクトルへ変換する（ あ ）層を用いて高速化を行う。また全ての単語候補から予測単語を選ぶ方法であれば最終層に（ い ）層を使うのが妥当だが、入力単語数を増加させると計算量が大きくなりおそくなるため、解決策として、最終層に（ う ）層を用いて、予測する単語が候補単語であるかどうかを二値で分類する手法を取ることがある。この手法は（ え ）とよばれ、利用する負例には出現頻度が（ お ）単語が優先的に選ばれる。

✓ (あ) の選択肢について、以下のうちから適切なものを1つ選べ。 * 1/1

- ☐ Softmax
- ☐ Sigmoid
- ☐ ReLU
- ☒ Embedding



✓ (い) の選択肢について、以下のうちから適切なものを1つ選べ。 * 1/1

- ☒ Softmax
- ☐ Embedding
- ☐ ReLU
- ☐ Sigmoid



✓ (う) の選択肢について、以下のうちから適切なものを1つ選べ。 * 1/1

- ☐ ReLU
- ☒ Sigmoid
- ☐ Softmax
- ☐ Embedding



✓ (え) (お) に当てはまる組み合わせについて、以下のうちから *1/1
適切なものを1つ選べ。

- ☒ (え) 負例サンプリング (お) 高い
- ☐ (え) 貪欲的サンプリング (お) 低い
- ☐ (え) 負例サンプリング (お) 低い
- ☐ (え) 貪欲的サンプリング (お) 高い



✓ 2018年にGoogleが発表した自然言語処理の教師なし事前学習モデルとし *1/1
て、BERT (Bidirectional Encoder Representations from Transformers)
がある。BERT が行なっている事前学習手法として、Next Sentence
Predictionがある。これは2つの文章を入力として、それぞれが連続して
いるかを予測させる事前学習手法である。BERT が他に行っている事前
学習手法について、最も適切なものを以下のうちから1つ選べ。

- ☒ Masked Language Model
- ☐ Conditional Text Generation
- ☐ Continuous Bag of Words
- ☐ Contrastive Pre-training



✓ 大規模な自然言語処理モデルの1つとして BERT（Bidirectional Encoder Representations from Transformers）がある。BERT における入力では通常の埋め込み手法に加えて、Segment Embedding と（あ）がある。Segment Embedding は、（い）をモデルに伝えるための埋め込み手法であり、（あ）は各トークンが文章のどこに配置されているかをモデルに伝えるための埋め込み手法である。

*1/1

（あ）（い）に当てはまる組み合わせについて、最も適切なものを以下のうちから1つ選べ。

- ☐ （あ）Entity Embedding （い）入力された文章が「質問文」であるかなどの文章の形式
- ☐ （あ）Entity Embedding （い）入力された各トークンが、それぞれどの文章に属しているか
- ☒ （あ）Position Embedding （い）入力された各トークンが、それぞれどの文章に属しているか ✓
- ☐ （あ）Position Embedding （い）入力された文章が「質問文」であるかなどの文章の形式

NLPの世界を大きく変えたモデルに、論文「Attention is all you need」で発表されたTransformerがある。Transformerは、それまでNLPの世界で主流であったRNN（及びCNN）を利用するモデル構造をやめ、Attentionのみを用いてEncoder-Decoder型のモデルを設計した。Transformer は当時の最高精度を更新し、NLPのデファクトスタンダードになった。

Transformerの成功により、Transformerを双方向RNNのように利用した（あ）がGoogleから発表され、大きな話題となった。（あ）の事前学習として（い）が（う）学習によって行われる。



✓ (あ) の選択肢について、以下のうちから適切なものを1つ選べ。 * 1/1

- ☐ ELMo
- ☐ GPT
- ☐ GNMT
- ☒ BERT



✓ (い) の選択肢について、以下のうちから適切なものを1つ選べ。 * 1/1

- ☒ 二値分類と多クラス分類
- ☐ 二値分類
- ☐ 回帰
- ☐ 多クラス分類



✓ (う) の選択肢について、以下のうちから適切なものを1つ選べ。 * 1/1

- ☐ 転移
- ☒ 教師なし
- ☐ 強化
- ☐ 教師あり



以下のコードを踏まえて、設問に答えよ。

```
from transformers import BertTokenizer, BertForMaskedLM,
BertForNextSentencePrediction
import torch

# 事前学習済みのBERTモデルとTokenizerをロードします。
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')
mlm_model = BertForMaskedLM.from_pretrained('bert-base-uncased')
nsp_model = BertForNextSentencePrediction.from_pretrained('bert-base-uncased')
# 利用するテキストを定義します。
text_a = "I enjoy walking with my cute dog."
text_b = "He's very playful."

# --- MLM (Masked Language Model) セクション ---

# ターゲットとなる単語をマスクします。
masked_index = 4
tokens = tokenizer.tokenize(text_a)
tokens[masked_index] = '[MASK]'
# 入力をエンコードし、テンソルに変換します。
encoded_input_mlm = tokenizer(tokens, (あ))
with torch.no_grad():
    output = mlm_model(**encoded_input_mlm)
    predictions = output.logits

# 予測されたトークンIDを取得し、トークンに変換します。
predicted_token_id = torch.argmax(predictions[0, masked_index]).item()
predicted_token = tokenizer.convert_ids_to_tokens([predicted_token_id])[0]

# --- NSP (Next Sentence Prediction) セクション ---

# 2つのテキストをエンコードし、テンソルに変換します。
encoded_input_nsp = tokenizer(text_a, text_b, (あ), truncation=True, padding=True,
max_length=512)
with torch.no_grad():
    output = nsp_model(**encoded_input_nsp)
    logits = output.logits

# 予測されたクラスを取得し、2つのセンテンスが連続しているかどうかを判断します。
is_next = torch.argmax(logits, dim=1).item() == 0
```

コード1 事前学習



✕ コード 1 のPytorchのプログラム内（ あ ）について、以下のうち正しい選択肢を 1 つ選べ。 *0/1

- ☐ return_tensors='np'
- ☐ return_tensors='pt'
- ☒ as_tensors=True
- ☐ return_tensors='tf'

✕



- ✓ transformers ライブラリを用いて、BERT の 'tokenizer' を使用すること *1/1
ができる。

コード2のプログラムの出力結果（ い ）は、BERT の 'tokenizer' を使用してトークナイズした結果である。出力結果（ い ）の内のトークンID「102」に対応するトークンとして正しいものを、以下の選択肢から1つ選べ。

```
from transformers import BertTokenizer

# BERTモデルのtokenizerをロード
tokenizer = BertTokenizer.from_pretrained('bert-base-uncased')

# トークン化するテキスト
text = "Hello, BERT! How are you?"

# トークンに分割
token = tokenizer.tokenize(text)
print(f"Token: {token}")

# トークンをIDに変換
token_ids = tokenizer.encode(token)
print(f"Token_IDs: {token_ids}")

# IDをトークンに変換
tokens = tokenizer.convert_ids_to_tokens(token_ids)
```

コード2 tokenizer

```
Token: ['hello', ',', 'bert', '!', 'how', 'are', 'you', '?']
Token_IDs: [101, 7592, 1010, 14324, 999, 2129, 2024, 2017, 1029, 102]... (い)
```

出力結果

- ☒ [SEP] ✓
- ☐ you
- ☐ ?
- ☐ [CLS]

✓ GPT-3のように、特定のタスクに特化せず、多様な下流のタスクに適用 *1/1
できるように後から調整できるモデルを（ あ ）という。（ あ ）で
は、特定のタスクに適用するために調整（アダプティング）を行うが、
GPT-3では入力にタスクの説明などを行うことによって様々なタスクに
適用することができる。この際、入力に用いられている文章などを（
い ）という。

（ あ ）（ い ）に当てはまる組み合わせについて、以下のうちから
適切なものを1つ選べ。

- ☐ （あ）一般モデル （い） プロンプト
- ☐ （あ）基盤モデル （い） クエリ
- ☒ （あ）基盤モデル （い） プロンプト
- ☐ （あ）一般モデル （い） クエリ



BERTの成功に引き続き、2018年に「Improving Language Understanding by Generative Pre-Training」という論文でGPTというモデルが発表された。GPTは、Transformer のスケーラビリティを最大限生かすことを目指し、シンプルに巨大化することで、性能の向上を目指した。その後、基本的にGPTを巨大化させる形で後継モデルとして、GPT-2、GPT-3が開発された。

GPT-3は、GPTの100倍のパラメータ数を持ち、飛躍的に性能が向上し、人間が書いた文章と見分けがつかないものが生成されたことで、大きな話題を呼んだ。また、プログラミングや計算など、明示的に学習させていないことも行うことができることが確認され、高い汎用性も評価された。



✕ GPT-2に関する説明文として誤っているものを選び。 *

0/1

- ☐ GPT-2は、発表当時のあらゆる自然言語処理タスクで最高性能を達成し、不得意なタスクが存在しないことが確認された。
- ☒ GPT-2は、次のトークンを予測する言語モデルとしてWebTextで事前学習され、タスク固有の学習を行わずに複数の自然言語処理タスクへ適用する能力が検証された。 ✕
- ☐ GPT-2の最大モデルは学習済みだったが、悪用への懸念から、当初は段階的に公開された。
- ☐ 最大のGPT-2は約15億のパラメータを持ち、初代GPTよりも10倍以上大きなモデルである。

✓ GPT-3に関する説明文として誤っているものを選び。 *

1/1

- ☒ GPT-3では、モデルを大きくしても、Few-shot による転移学習の効果は変わらないことが確認された。 ✓
- ☐ ウェブデータのクローリングなどでデータセットを作成しているため、データの中身を精査することが難しく、高品質なデータセットとの類似度などを基準として、学習データをフィルタリングしている。
- ☐ 短い文章であればクオリティが高いが、文章が長くなると一貫性を失うなどの問題が指摘された。
- ☐ 大規模モデルにあわせて、大規模データセットが作成されたが、ベンチマークデータとの重複を調査し、データ汚染の影響を検証する処理が行われた。

- ✓ 事前学習を行ったモデルに対して（ あ ）を実施する場合、非常に少 *1/1
数のラベル付きデータを使ってモデルを新たなタスクに適応させること
を考える。これは本来（ い ）などの問題がある場合において適用さ
せる手法であるが、GPT-3などの大規模言語モデルの場合はプロンプト
に回答例などを加えることで、（ あ ）を実行できる。

（ あ ）（ い ）に当てはまる組み合わせについて、以下のうちから
適切なものを1つ選べ。

- ☐ （あ） Some Shot Learning （い） ラベル付けが困難・高コスト
- ☐ （あ） Few Shot Learning （い） モデルの学習進行が遅い
- ☐ （あ） Some Shot Learning （い） モデルの学習進行が遅い
- ☒ （あ） Few Shot Learning （い） ラベル付けが困難・高コスト ✓

- ✓ プロンプトベースラーニングは、適切なプロンプトを設計してモデルに *1/1
入力することに焦点を当てた学習手法であり、モデルのファインチュー
ニングではなく、このプロンプトの設計と入力が必要な役割を果たす。
大規模言語モデルにおけるプロンプトベースラーニングについて、以下
のうちから適切なものを1つ選べ。

- ☐ プロンプトベースラーニングでは、大規模言語モデルのファインチューニングを通
じて、モデルの性能を向上させることが主な目的である。この手法は、特定のタス
クに対するモデルの反応を改善するために使用される。
- ☐ プロンプトベースラーニングは、モデルに新しいプログラミング言語を教えるため
に使用される技術である。この方法では、モデルが異なるプログラミング言語で
書かれたコードを理解し、実行する能力を高める。
- ☒ プロンプトベースラーニングは、大規模言語モデルを使用する際に適切なプロ ✓
ンプトを設計し入力することでモデルの反応を最適化する方法である。このア
プローチは、モデルの内部構造を改変することなく、特定の応答を引き出すた
めに使用される。
- ☐ プロンプトベースラーニングでは、モデルの学習データセットを変更することで、
特定の応答を得ることができる。この方法は、モデルが新しいデータタイプに適
応するのを助けるために用いられる。

✓ 基盤モデルは、多様なデータに基づいて訓練され、さまざまなタスクに応用可能である。特に、GPTシリーズのようなモデルは、その柔軟性と応用範囲の広さで注目を集めている。基盤モデルについて、以下のうちから適切なものを1つ選べ。 *1/1

- ☒ 基盤モデルは、入力されるプロンプトに基づいて様々なタスクを解くことが可能なモデルで、GPTシリーズのようにファインチューニングを含む調整により多様なタスクに対応する。 ✓
- ☐ 基盤モデルは、特定のタスクに特化したモデルであり、GPTシリーズではプロンプトによる指示に基づいて限定的なタスクのみを解くことができる。
- ☐ 基盤モデルは、入力されるプロンプトに関係なく、あらかじめ定義されたタスクのみを解くことができるモデルで、GPTシリーズの特徴とされている。
- ☐ 基盤モデルは、GPT 2以降のみに存在し、それ以前のモデルはファインチューニングによる調整が主な特徴である。

✓ GPTをはじめとする生成AIの欠点を補うために、RAG (Retrieval-Augmented Generation) と呼ばれる技術が様々なアプリケーションで採用されている。次のうち、RAGを適用するユースケースとして最も不適切なものを1つ選べ。 *1/1

- ☐ 社内のナレッジベースや過去の議事録を検索して、担当者の質問に回答する
- ☒ 会議音声文字起こし後、誤字脱字や不自然な表現を修正して読みやすい文章に整えるために、外部文書を検索しながら推敲する ✓
- ☐ 自社製品のマニュアルを参照させたいうえで、カスタマーサポートのユーザーからの問い合わせに回答する
- ☐ 医療ガイドラインなど、定期的に更新される専門文書の最新版を参照しながら回答を生成する

✓ RNNやTransformerなどに基づく自己回帰生成モデルは、過去系列から次時刻の値を生成するモデルである。自己回帰生成モデルの訓練および推論に関する説明として、最も適切な選択肢を1つ選べ。 *1/1

- ☒ 訓練時には真の過去系列を教師強制として入力するが、推論時には自身の過去の出力を次時刻の入力として用いる ✓
- ☐ 訓練時も推論時も、常に真の過去系列を教師強制として入力する
- ☐ 訓練時には自身の過去の出力を入力として用い、推論時には真の過去系列を教師強制として入力する
- ☐ 訓練時も推論時も、常に自身の過去の出力を次時刻の入力として用いる

✗ 生成モデルは最終的に計算される事後確率を用いて分類を行う際は識別モデルと変わらない。また、生成モデルの場合は（ あ ）といったメリットがある。生成モデルは、疑似データの生成や異常検知への応用、半教師あり学習や欠損値への対応が比較的容易など様々な可能性がある。 *0/1

（ あ ）の選択肢について、以下のうちから適切なものを1つ選べ。

- ☐ モデルの出力の解釈や学習とは別に確率に基づいた識別境界を設定することが可能になる。
- ☒ 入力データ空間に識別超平面を描くことができ、入力データ空間内のクラス分布が直感的に把握できる。 ✗
- ☐ 入力データの生成過程やクラスごとのデータ分布などを知ることができる。
- ☐ 数学的な操作が容易く、データ量が少ない場合でも機能する。

✓ 生成モデルと識別モデルに関する説明として適切でないものを1つ選べ。 *1/1

- ☒ パターン認識は、識別モデルを使う必要がある。 ✓
- ☐ 識別モデルは、データを識別するための境界線を訓練データから直接的に求めようとする。
- ☐ 新たなデータを作成したい場合などは生成モデルを使う必要がある。
- ☐ 生成モデルは、分類対象となるデータがどのような確率モデルから生成されたかをモデル化し、そのモデルに基づいて分類を行う。

✓ 生成モデルに関する説明として、次の選択肢の中から適切でないものを1つ選べ。 *1/1

- ☐ 拡散モデルは、深層生成モデルの一種である。ノイズ（潜在変数）からデータを学習することは難しいことから、データからノイズへ拡散させる過程を考え、この拡散の逆方向にたどることによってデータを生成する。
- ☐ 深層生成モデルの一つに、変分自己符号化器がある。変分自己符号化器は生成データと学習データの分布間距離を測り、その距離が最小になるように学習する。
- ☒ 生成データと学習データの分布間距離を厳密に計算することは難しいため、Flow-based modelや自己回帰型生成モデルは、分布間距離の上限値で代用している。 ✓
- ☐ 敵対的生成ネットワークは深層生成モデルの一つである。敵対的生成ネットワークは、生成器と識別器が敵対者との競争下に置かれるゲーム理論的状况に基づいている。

- ✓ 分類問題における機械学習モデルは、「パターン認識と機械学習」 [Bishop, 2006]によると、モデル化する対象を基準に大きく3つのモデルを考えることができる。 *1/1

まず、入力データからそれが属するラベルを直接出力する関数を識別関数と呼ぶ。例えば、サポートベクトルマシンがこれに当たる。

次に、ラベルを直接出力するのではなく、入力データが各クラスラベルに属する確率をモデル化する手法を識別モデルと呼ぶ。分類問題におけるニューラルネットワークがこれに当たる。

識別モデルを用いると、識別関数と異なり、(あ) といったメリットがある。

(あ) に入る以下の文のうち、最も適切なものを1つ選べ。

- ☐ 入力データ空間に識別超平面を描くことができ、入力データ空間内のクラス分布が直感的に把握できる。
- ☐ 数学的な操作が容易く、データ量が少ない場合でも機能する。
- ☐ 入力データの生成過程やクラスごとのデータ分布などを知ることができる。
- ☒ 各クラスに属する確率を出力できるため、用途や損失に応じて意思決定の閾値を設定できる。 ✓

- ✓ 機械学習におけるモデルには、大きく分けて識別モデルと生成モデルの二種類がある。識別モデルと生成モデルの特徴及びモデル化の過程について、以下の選択肢の中から正しいものを1つ選べ。 *1/1

- ☐ サポートベクトルマシンのような識別関数は、クラスを直接推定する機能を持つが、識別モデルは入力に対する各クラスの確率を計算することはできない。
- ☐ 識別モデルは、入力データに対して各クラスの確率を計算し、最終的なクラス決定のためにはロジスティック回帰を用いるが、確率的な予測を行うことはない。
- ☒ 生成モデルは、各クラスごとに入力の確率分布を考慮し、ベイズの定理を用いて入力に対する各クラスの確率を求め、入力データが生成される確率的なプロセスをモデル化する。 ✓
- ☐ 識別モデルは入力データのクラスを確率的に予測し、生成モデルは各クラスの確率分布を計算するが、入力データが生成される確率的なプロセスをモデル化することはできない。

✓ 拡散モデルは入力データに対して逐次的にガウシアンノイズを加えることで、完全なノイズデータを作成するモデルである。このモデルはノイズデータから新しいデータを生成する能力を持つため、様々なアプリケーションで使用されている。拡散モデルに関する以下の記述について、正しいものを1つ選べ。 *1/1

- ☐ 拡散モデルの主要な機能は、ノイズとクラスや文章を指定することで、それらに従った新しいデータを生成することである。
- ☐ DDPMは、拡散モデルの一つとして知られ、スペルチェックや音声合成にも使用される。
- ☒ 拡散モデルは、初めに完全なノイズデータを作成し、その後ノイズを徐々に取り除くことで元のデータを再構築する。 ✓
- ☐ 拡散モデルは、ノイズを追加する過程のみを学習し、ノイズの除去は行わない。

✓ フローベース生成モデルは、単純な確率分布に一連の逆変換可能な変換を適用することで、複雑なデータ分布を表現する生成モデルである。変数変換の公式とヤコビアン行列式を用いることで、データの尤度を計算できる。また、その過程で変数変換やヤコビアン行列式が用いられることも特徴的である。フローベース生成モデルに関する以下の記述について、正しくないものを1つ選べ。 *1/1

- ☐ フローベース生成モデルは、用いる変換関数が必ず逆変換可能でなければならない。
- ☐ フローベース生成モデルの損失関数は、負の対数尤度であり、これによりデータの分布を直接変換する特性を持つ。
- ☒ フローベース生成モデルでは、使用できる変換関数の個数が理論上1つに制限されているため、複数の変換を組み合わせることはできない。 ✓
- ☐ 潜在変数 z_{i-1} を z_i に変換する過程で、変換後の確率分布は変換前の確率分布とヤコビアン行列式によって表現される。

✓ 拡散モデルとフローベース生成モデルのアルゴリズムおよび用途について、以下の選択肢の中から正しいものを1つ選べ。 *1/1

- ☐ 拡散モデルは、最初にクリアな画像を生成し段階的にノイズを加えることで高品質な画像を生成する。フローベース生成モデルは非可逆変換によって複雑な分布を形成し、データの生成と推論が難しいため、主に高速な画像生成に適している。
- ☒ 拡散モデルは、画像に段階的にノイズを付与し、その逆プロセスによって高品質な画像を生成する。フローベース生成モデルは、一連の可逆変換を用いて複雑な確率分布を形成し、変換の逆方向を容易に計算できるため、効率的なデータ生成と推論に適している。 ✓
- ☐ 拡散モデルは、画像からノイズを除去することで高品質な画像を生成するが、生成プロセスが時間を要するため、高速な画像生成には不向きである。一方、フローベース生成モデルは、非可逆変換を用いて複雑な確率分布を形成し、高速なデータ生成に適している。
- ☐ 拡散モデルは、ノイズを段階的に除去してクリアな画像を再構築するが、生成プロセスは効率が低い。フローベース生成モデルは、可逆変換を用いて確率分布を形成し、変換の逆方向を計算しにくいいため、主に複雑なデータ分析に使用される。

✓ MNISTの手書き数字データセットを、クラスラベルを使用しない標準的なVAEで学習し、2次元の潜在空間を可視化した。一般的に期待される傾向として、最も適切なものを1つ選べ。 *1/1

- ☒ 形状の似た入力 は 潜在空間上で近い領域に配置されやすく、クラス間に滑らかな遷移や一部の重なりが見られる場合がある。 ✓
- ☐ 数字の各クラスは必ず完全に分離され、異なるクラス同士が重なることはない。
- ☐ 数字としての値が近いクラスほど、必ず潜在空間上でも近くに配置される。
- ☐ 潜在変数は入力画像の特徴と無関係に、完全にランダムな位置へ配置される。

✓ VAE（変分自己符号化器）の学習時に、潜在空間のサンプリングを行う際の勾配の伝播を可能にするための手法として、リパラメタリゼーショントリックがある。リパラメタリゼーショントリックについての説明文として、以下のうちから最適なものを1つ選べ。 *1/1

- ☐ 潜在変数 z は、 $z=\mu/\sigma*\varepsilon$ の形式でパラメタリゼーションされ、 ε は均一分布からサンプリングされる。この方法により、潜在空間の任意の点でのサンプリングが容易になる。
- ☐ リパラメタリゼーショントリックは、潜在変数 z をサンプリングする際に使用される確率分布のパラメータを直接学習することで、勾配の伝播を回避するテクニックである。
- ☐ ε は標準正規分布からサンプリングされ、 $z=\mu*\varepsilon+\sigma$ としてパラメタリゼーションされる。この方式は、潜在変数の再構成エラーを最小化するために導入された。
- ☒ 潜在変数 z は、 $z=\mu+\varepsilon\sigma$ の形式でパラメタリゼーションされ、 ε は標準正規分布からサンプリングされる。これにより、勾配は μ と σ を通して直接伝播する。 ✓

データの解析を行う際に、対象のデータの特徴を把握する目的・方法として、次元削減をおこなうことがある。データの次元数はデータの特徴の数ととらえられる。次元数を減らすことでデータを扱いやすくしたり、可視化することが可能になる。

次元削減の具体的な手法としてはPCA（主成分分析）などが挙げられるが、ニューラル・ネットワークを用いた機械学習を使った次元削減の手法として、（あ）がよく知られている。（あ）の用途としては、（い）などがある。

✓ （あ）の選択肢について、以下のうちから適切なものを1つ選べ。 * 1/1

- ☐ 独立成分分析
- ☐ isomap
- ☐ 制約ボルツマンマシン
- ☒ オートエンコーダ ✓

✓ (い) の選択肢について、以下のうちから適切でないものを1つ選べ。 *1/1

- ☐ 次元削減
- ☐ 再構成誤差を利用した異常検知
- ☒ 正解ラベルを自動生成し、ラベル付き教師データを作成すること ✓
- ☐ 特徴抽出やニューラルネットワークの事前学習

✓ 生成モデルの1つであるVAEは (あ) 的な (い) を学習する。VAEは、入力を受けて出力が決定論的に決まるオートエンコーダと異なり、確率分布を導入することで確率的にデータを生成する。 *1/1

(あ) (い) に当てはまる組み合わせについて、最も適切なものを以下のうちから1つ選べ。

- ☐ (あ) 連続 (い) 顕在空間
- ☒ (あ) 連続 (い) 潜在空間 ✓
- ☐ (あ) 離散 (い) 潜在空間
- ☐ (あ) 離散 (い) 顕在空間

✓ VAEの説明文について、以下のうちから不適切な説明を一つ選べ。 * 1/1

- ☐ VAEのエンコーダは、入力データに対応する潜在変数の近似事後分布のパラメータを推定する。
- ☒ VAEは確率分布からサンプリングを行うため、誤差逆伝播を使用してパラメータを更新することはできない。 ✓
- ☐ VAEでは、再構成に関する項と、近似事後分布を事前分布へ近づけるKLダイバージェンス項を含む目的関数を用いる。
- ☐ 再パラメータ化トリックを利用することで、確率的なサンプリングを含むモデルでも誤差逆伝播を用いて学習できる。

✓ 敵対的生成ネットワーク（GAN：Generative Adversarial Network）は、 *1/1 ニューラルネットワークを使用した生成モデルである。GANは、生成器（Generator）と識別器（Discriminator）の2つの部分から構成されている。GANに発生する問題の1つであるモード崩壊について、以下のうちから最適なものを1つ選べ。

- ☐ GANが大量のデータを必要とする現象。
- ☒ 生成ネットワークが常に同じ、あるいは非常に似たデータしか生成しなくなる現象。 ✓
- ☐ GANの学習が非常に速く終了する現象。
- ☐ 識別ネットワークが生成データを全く識別できなくなる現象。

- ✓ GAN（敵対的生成ネットワーク）における問題の1つとしてモード崩壊 *1/1がある。Wasserstein GAN（WGAN）はモード崩壊と学習の安定化を目指した生成モデルであり、WGANの目的関数は元のGANとは異なる損失関数を使用する。WGANの目的関数について、以下のうちから適切なものを1つ選べ。

$$\max_D E_{x \sim P_{data}(x)} [D(x)] - E_{z \sim P_{data}(z)} [D(G(z))]$$

☒ A


$$\min_D E_{x \sim P_{data}(x)} [1 - D(x)] - E_{z \sim P_{data}(z)} [1 - D(G(z))]$$

☐ B

$$\max_G \min_D E_{x \sim P_{data}(x)} [D(x)] - E_{z \sim P_{data}(z)} [D(G(z))]$$

☐ C

$$\max_D \min_G E_{x \sim P_{data}(x)} [\log D(x)] - E_{z \sim P_{data}(z)} [1 - \log D(G(z))]$$

☐ D

2014年に発表され、現在生成モデルとして有力なベースモデルとなっているのが、GANである。GANは、Generator と Discriminatorが互いに競争するように学習していくことで、より高品質なものを生成しようとする。

GANに関して、最適な生成器と識別器の両方が得られた場合、識別器の出力は（ あ ）となる。なお、モード崩壊を引き起こすなど（ い ）が未解決問題であることが知られており、様々な工夫が行われている。



✓ (あ) の選択肢について、以下のうちから適切なものを1つ選べ。 * 1/1

☐ $+\infty$

☐ 1

☒ 0.5

☐ $-\infty$



✓ (い) の選択肢について、以下のうちから適切なものを1つ選べ。 * 1/1

☐ モデルの解釈性

☐ 生成の高速化

☐ 軽量化

☒ 学習の安定化



✕ GANの損失関数として最も適切な式を1つ選べ。ただし、左辺は以下の *0/1式で始まるものとする。

$$\min_G \max_D V(D, G) =$$

$$E_{x \sim P_{data}(x)} [\log D(x)] \\ + E_{z \sim P_{data}(z)} [\log (1 - D(G(z)))]$$

☐ C

$$E_{x \sim P_{data}(x)} [\log D(x)] \\ + E_{z \sim P_{data}(z)} [\log D(G(z))]$$

☒ A

✕

$$E_{x \sim P_{data}(x)} [\log D(x)] \\ + E_{z \sim P_{data}(z)} [\log G(x)]$$

☐ B

$$E_{x \sim P_{data}(x)} [\log D(x)] \\ + E_{z \sim P_{data}(x)} [\log (1 - D(G(x)))]$$

☐ D

✓ GANに関して、次の選択肢から最も適切なものを1つ選べ。 *

1/1

- ☐ Discriminatorは、データセットと同じような画像を生成する。GeneratorはGeneratorで生成した画像を製錬した画像を出力する。
- ☐ Discriminatorは、データセットと同じような画像を生成する。Generatorは入力画像がデータセットの中にある本物の画像かDiscriminatorが生成した偽物の画像かどうか判定する。
- ☐ Generatorは、データセットと同じような画像を生成する。DiscriminatorはGeneratorで生成した画像を精錬した画像を出力する。
- ☒ Generatorは、データセットと同じような画像を生成する。Discriminatorは入力画像がデータセットの中にある本物の画像かGeneratorが生成した偽物の画像かどうかを判定する。 ✓

✓ 訓練データを学習し、それらのデータと類似した新しいデータを生成するモデルのことを生成モデルという。つまり、訓練データの分布と生成データの分布が一致するように学習していくモデルである。 *1/1

中でも、「訓練データから新しいデータを生成するネットワーク」と「与えられたデータが訓練データであるか、モデルが生成したデータであるかを判別するネットワーク」の2つを交互に学習させていくようなモデルを（ あ ）という。

（ あ ）の選択肢について、以下のうちから最も適切なものを1つ選べ。

- ☐ Support Vector Machine
- ☒ Generative Adversarial Nets ✓
- ☐ Residual Network
- ☐ Restricted Boltzmann Machine

- ✓ GANは優れた生成能力を持つが、学習が難しいという問題がある。GAN ^{*1/1}の枠組みをCNNに適用した深層畳み込みGANでは、学習を安定化させるためのテクニックとして（ あ ）ことが提案された。これを契機に、CNNを使用したDCGANなどが提案された。

また、GANの派生モデルも数多く作成され、ある程度生成する画像をコントロールできるGANの一種であるConditionalGANなども開発されている。

（ あ ）の選択肢について、以下のうちから最も適切ではない選択肢を1つ選べ。

- ☒ 生成器のみバッチ正規化を使用する。 ✓
- ☐ 生成器の活性化関数は、最終層にtanh、それ以外にはReLUを使用する。
- ☐ プーリングの代わりに、ストライドを2以上に設定した畳み込み層や転置畳み込み層を用いる。
- ☐ 識別器の活性化関数にLeaky ReLUを使用する。

- ✓ ConditionalGANについて、正しい説明を1つ選べ。 *

1/1

- ☐ DCGANのような従来のGANにおいては、Discriminatorは入力された画像が学習用データのものか、Generatorから生成されたものかを識別するように学習するが、ConditionalGANでは、Discriminatorは学習用データの正解ラベルを上手く識別できるように学習する。
- ☒ Conditional GANは、クラスラベルなどの条件情報を生成器と識別器へ与えることで、指定した条件に対応するデータを生成できるようにする。DCGANなど従来のGANは条件データを付与していない。 ✓
- ☐ ConditionalGANでは、Generatorの画像生成時に与えられる潜在変数zに明示的に制約条件を加えることによって、ConditionではないGANよりも生成される画像の質を向上させる手法である。
- ☐ DCGANのような従来のGANについても、生成する画像を明示的に指定することができたが、ConditionalGANを使うことで、Generatorが生成する画像の質が向上することが知られている。

- ✓ Conditional GANの損失関数として最も適切な式を1つ選べ。ただし、*1/1
左辺は以下の式で始まるものとする。

$$\min_G \max_D V(D, G) =$$

$$E_{x \sim P_{data}(x)} [\log D(x|y)] \\ + E_{z \sim P_{data}(z)} [\log (1 - D(G(z|y)))]$$

☒ A


$$E_{x \sim P_{data}(x)} [\log D(x)] \\ + E_{z \sim P_{data}(z)} [\log (1 - D(G(x)))]$$

☐ B

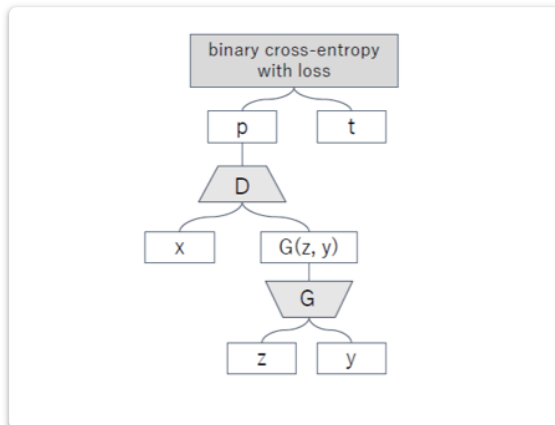
$$E_{x \sim P_{data}(x)} [\log D(x|y)] \\ + E_{z \sim P_{data}(z)} [\log (1 - D(G(z)))]$$

☐ C

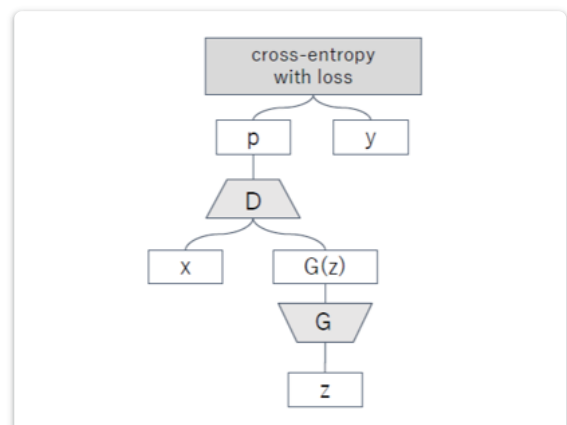
$$E_{x \sim P_{data}(x)} [\log D(y)] \\ + E_{z \sim P_{data}(z)} [\log (1 - D(G(z|y)))]$$

☐ D

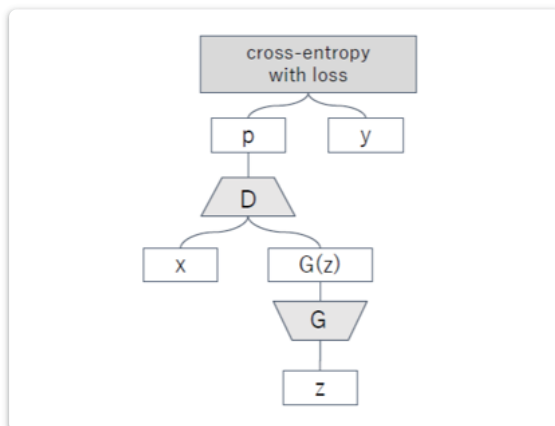

- ✓ Conditional GANのネットワーク構造の例として適切なものを選び。ただし、ネットワークはDiscriminatorの学習時のものとする。



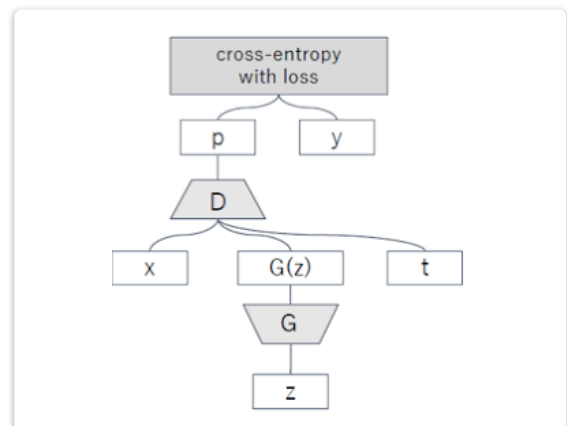
☒ A



☐ B



☐ C



☐ D

このフォームは デロイト トーマツ ディープスクエア株式会社 内部で作成されました。
このフォームが不審だと思われる場合 [報告](#)

Google フォーム



